

```
...     spam = ['cat', 'dog', 'moose']
...
>>> someFunction()
someFunction() called.
```

When `someFunction()` is called, Python allocates memory for the list `['cat', 'dog', 'moose']`. The programmer doesn't need to figure out how many bytes of memory to request because Python manages this automatically. Python's garbage collector will free the local variables when the function call returns to make that memory available for other data. Garbage collection makes programming much easier and less bug-prone.

Literals

A *literal* is text in the source code for a fixed, typed-out value. In the following code example

```
>>> age = 42 + len('Zophie')
```

the `42` and `'Zophie'` text are integer and string literals. Think of a literal as a value that literally appears in source code text. Only the built-in data types can have literal values in Python source code, so the variable `age` isn't a literal value. [Table 7-1](#) lists some example Python literals.

[Table 7-1](#): Examples of Literals in Python

Literal	Data type
42	Integer
3.14	Float
1.4886191506362924e+36	Float
"""Howdy!"""	String
r'Green\Blue'	String
[]	List
{'name': 'Zophie'}	Dictionary
b'\x41'	Bytes
True	Boolean
None	NoneType